

ASSIGNMENT:- 4

AIM:- MongoDB CRUD Operations

PROBLEM STATEMENT:- Creation of database and CRUD Operations: Insert, Update and Delete Document and Query Operations.

THOERY:-

Introduction to MongoDB

MongoDB is a NoSQL (Non-relational), document-oriented database developed by MongoDB Inc. It is designed for ease of development and scaling, and is used for handling large volumes of unstructured or semi-structured data. Unlike traditional Relational Database Management Systems (RDBMS) that store data in rows and tables, MongoDB stores data in JSON-like documents, which makes it highly flexible.

Documents

- Documents are the basic unit of data in MongoDB. Each document is a set of key-value pairs.
- Documents are similar to JSON (JavaScript Object Notation), but use a binary format called BSON (Binary JSON), which allows for more data types and efficient storage.

Example of a document:

```
{
  "name": "John",
  "age": 21,
  "course": "Data Science",
  "skills": ["Python", "MongoDB", "Machine Learning"]
}
```

MongoDB creates a database implicitly when data is inserted. To switch to a database:

```
use database_name;
```

Collections

- A collection is a group of MongoDB documents. Collections are similar to tables in RDBMS.
- Unlike SQL tables, documents in a collection do not require a fixed schema, meaning each document can have different fields.

A collection is created when a document is inserted or explicitly using:

```
db.createCollection("collection_name");
```

Feature	MongoDB	RDBMS (e.g., MySQL, PostgreSQL)
Data Storage	Stores data as documents in BSON/JSON-like format. Each document is a self-contained unit.	Stores data in tables with rows and columns.
Schema	Dynamic schema: No need to define the structure in advance. Fields can differ per document.	Fixed schema: Must define the table structure before inserting data.
Joins	Supports limited joins using <code>\$lookup</code> , but it's not as native or powerful as in SQL.	Fully supports JOIN operations across multiple tables.
Scalability	Horizontally scalable via sharding (data is distributed across multiple servers).	Typically vertically scalable (scale by upgrading hardware).
Transactions	Supports multi-document ACID transactions since v4.0.	Fully supports ACID-compliant transactions across multiple tables.
Use Case Fit	Great for unstructured, semi-structured, and rapidly changing data.	Best for structured data, strong relational logic, and complex queries.

CRUD Operations in MongoDB

CRUD stands for Create, Read, Update, and Delete — the four fundamental operations performed on any data in a database. MongoDB provides simple and powerful methods to perform these operations on its document-based collections.

1. **Create – Insert Data:-** MongoDB uses the `insertOne()` and `insertMany()` methods to add documents into a collection.

1. **`insertOne()`:-** Used to insert a single document into a collection.

Syntax:

```
db.collection_name.insertOne({key1: value1, key2: value2, ...});
```

2. **`insertMany()`:-** Used to insert multiple documents at once.

Syntax:

```
db.collection_name.insertMany([{...}, {...}, ...]);
```

2. **Read – Query Data:-** To retrieve data from MongoDB, the `find()` method is used.

1. **`find()`:-** Fetches documents from a collection. If no condition is provided, it returns all documents.

Examples:

- `db.products.find();` // Fetches all documents
- `db.products.find({name: "Notebook"});` // Fetches documents with name = "Notebook"

3. **Update – Modify Data:-** `updateOne()` or `updateMany()` functions help in modifying one or more documents.

1. **`updateOne()`:-** Updates the first matching document.

Syntax:

```
db.collection_name.updateOne(filter, update_operation);
```

2. updateMany():- Updates all documents that match the condition.

Syntax:

```
db.collection_name.updateMany(filter, update_operation);;
```

4. Delete – Remove Data:- Delete data using deleteOne() or deleteMany().

1. deleteOne():- Removes the first document that matches the condition.

Example:

```
db.products.deleteOne({name: "Laptop"});
```

2. deleteMany():- Deletes all documents that match the condition.

Examples:

- `db.products.deleteMany({category: "Books"});` // Delete all books
- `db.products.deleteMany({});` // Deletes ALL documents from the collection!

5. Query Operators in MongoDB

MongoDB offers a wide range of comparison and logical operators to build complex queries:

1. Comparison Operators

Operator	Description
<code>\$eq</code>	Equals
<code>\$ne</code>	Not equal
<code>\$gt</code>	Greater than
<code>\$lt</code>	Less than
<code>\$gte</code>	Greater than or equal to
<code>\$lte</code>	Less than or equal to

2. Logical Operators

Operator	Description
<code>\$and</code>	Logical AND
<code>\$or</code>	Logical OR
<code>\$not</code>	Negates a condition
<code>\$nor</code>	Opposite of OR (none must match)

CODE and OUTPUT:-

```
test> show dbs;
admin      40.00 KiB
config     48.00 KiB
local     104.00 KiB
practice  304.00 KiB

test> use test;
already on db test

test> db.createCollection("products");
{ ok: 1 }

test> db.products.insertOne({name: 'Laptop', category: 'Electronics', price: 40000});
{
  acknowledged: true,
  insertedId: ObjectId('67fdeee3aa4d1653a24aa58c')
}
```

```

test> db.products.insertMany([
...   {name: 'Smartphone', category: 'Electronics', price: 10000},
...   {name: 'Books', category: 'Stationary', price: 500}
... ]);
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('67fdeeedaa4d1653a24aa58d'),
    '1': ObjectId('67fdeeedaa4d1653a24aa58e')
  }
}

test> db.products.insertMany([
...   {name: 'Headphones', category: 'Tech Gadgets', price: 2500},
...   {name: 'Blender', category: 'Home Appliances', price: 3000},
...   {name: 'Notebook', category: 'Stationary', price: 50},
...   {name: 'T-shirt', category: 'Fashion', price: 800},
...   {name: 'Gaming Mouse', category: 'Tech Gadgets', price: 1500},
...   {name: 'Desk Lamp', category: 'Home Decor', price: 1200},
...   {name: 'Backpack', category: 'Accessories', price: 1800},
...   {name: 'Shoes', category: 'Fashion', price: 2500},
...   {name: 'Water Bottle', category: 'Utilities', price: 300},
...   {name: 'Tablet', category: 'Tech Gadgets', price: 20000}
... ]);
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('67fdef18aa4d1653a24aa58f'),
    '1': ObjectId('67fdef18aa4d1653a24aa590'),
    '2': ObjectId('67fdef18aa4d1653a24aa591'),
    '3': ObjectId('67fdef18aa4d1653a24aa592'),
    '4': ObjectId('67fdef18aa4d1653a24aa593'),
    '5': ObjectId('67fdef18aa4d1653a24aa594'),
    '6': ObjectId('67fdef18aa4d1653a24aa595'),
    '7': ObjectId('67fdef18aa4d1653a24aa596'),
    '8': ObjectId('67fdef18aa4d1653a24aa597'),
    '9': ObjectId('67fdef18aa4d1653a24aa598')
  }
}

test> db.products.find();
[
  {
    _id: ObjectId('67fdeee3aa4d1653a24aa58c'),
    name: 'Laptop',
    category: 'Electronics',
    price: 40000
  },
  {
    _id: ObjectId('67fdeeedaa4d1653a24aa58d'),
    name: 'Smartphone',
    category: 'Electronics',
    price: 10000
  },
  {
    _id: ObjectId('67fdeeedaa4d1653a24aa58e'),
    name: 'Books',
    category: 'Stationary',
    price: 500
  },
  {
    _id: ObjectId('67fdef18aa4d1653a24aa58f'),
    name: 'Headphones',
    category: 'Tech Gadgets',
    price: 2500
  },
  {
    _id: ObjectId('67fdef18aa4d1653a24aa590'),
    name: 'Blender',
    category: 'Home Appliances',
    price: 3000
  },
]

```

```

{
  _id: ObjectId('67fdef18aa4d1653a24aa591'),
  name: 'Notebook',
  category: 'Stationary',
  price: 50
},
{
  _id: ObjectId('67fdef18aa4d1653a24aa592'),
  name: 'T-shirt',
  category: 'Fashion',
  price: 800
},
{
  _id: ObjectId('67fdef18aa4d1653a24aa593'),
  name: 'Gaming Mouse',
  category: 'Tech Gadgets',
  price: 1500
},
{
  _id: ObjectId('67fdef18aa4d1653a24aa594'),
  name: 'Desk Lamp',
  category: 'Home Decor',
  price: 1200
},
{
  _id: ObjectId('67fdef18aa4d1653a24aa595'),
  name: 'Backpack',
  category: 'Accessories',
  price: 1800
},
{
  _id: ObjectId('67fdef18aa4d1653a24aa596'),
  name: 'Shoes',
  category: 'Fashion',
  price: 2500
},
{
  _id: ObjectId('67fdef18aa4d1653a24aa597'),
  name: 'Water Bottle',
  category: 'Utilities',
  price: 300
},
{
  _id: ObjectId('67fdef18aa4d1653a24aa598'),
  name: 'Tablet',
  category: 'Tech Gadgets',
  price: 20000
}
]

```

```
test> db.products.find({name: 'Books'});
```

```

[
  {
    _id: ObjectId('67fdeeedaa4d1653a24aa58e'),
    name: 'Books',
    category: 'Stationary',
    price: 500
  }
]

```

```
test> db.products.find({price: {$lt: 20000}});
```

```

[
  {
    _id: ObjectId('67fdeeedaa4d1653a24aa58d'),
    name: 'Smartphone',
    category: 'Electronics',
    price: 10000
  },
  {
    _id: ObjectId('67fdeeedaa4d1653a24aa58e'),
    name: 'Books',
    category: 'Stationary',
  }
]

```

```

    price: 500
  },
  {
    _id: ObjectId('67fdef18aa4d1653a24aa58f'),
    name: 'Headphones',
    category: 'Tech Gadgets',
    price: 2500
  },
  {
    _id: ObjectId('67fdef18aa4d1653a24aa590'),
    name: 'Blender',
    category: 'Home Appliances',
    price: 3000
  },
  {
    _id: ObjectId('67fdef18aa4d1653a24aa591'),
    name: 'Notebook',
    category: 'Stationary',
    price: 50
  },
  {
    _id: ObjectId('67fdef18aa4d1653a24aa592'),
    name: 'T-shirt',
    category: 'Fashion',
    price: 800
  },
  {
    _id: ObjectId('67fdef18aa4d1653a24aa593'),
    name: 'Gaming Mouse',
    category: 'Tech Gadgets',
    price: 1500
  },
  {
    _id: ObjectId('67fdef18aa4d1653a24aa594'),
    name: 'Desk Lamp',
    category: 'Home Decor',
    price: 1200
  },
  {
    _id: ObjectId('67fdef18aa4d1653a24aa595'),
    name: 'Backpack',
    category: 'Accessories',
    price: 1800
  },
  {
    _id: ObjectId('67fdef18aa4d1653a24aa596'),
    name: 'Shoes',
    category: 'Fashion',
    price: 2500
  },
  {
    _id: ObjectId('67fdef18aa4d1653a24aa597'),
    name: 'Water Bottle',
    category: 'Utilities',
    price: 300
  }
]

```

```

test> db.products.updateOne(
...   {name: 'Laptop'},
...   {$set: {price: 70000}}
... );
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}

```

```

test> db.products.find({name: 'Laptop'});
[

```

```

    {
      _id: ObjectId('67fdeee3aa4d1653a24aa58c'),
      name: 'Laptop',
      category: 'Electronics',
      price: 70000
    }
  ]

test> db.products.updateMany(
...   {category: 'Electronics'},
...   {$set: {category: 'Tech Gadgets'}}
... );
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 2,
  modifiedCount: 2,
  upsertedCount: 0
}

test> db.products.find({category: 'Tech Gadgets'});
[
  {
    _id: ObjectId('67fdeee3aa4d1653a24aa58c'),
    name: 'Laptop',
    category: 'Tech Gadgets',
    price: 70000
  },
  {
    _id: ObjectId('67fdeeedaa4d1653a24aa58d'),
    name: 'Smartphone',
    category: 'Tech Gadgets',
    price: 10000
  },
  {
    _id: ObjectId('67fdef18aa4d1653a24aa58f'),
    name: 'Headphones',
    category: 'Tech Gadgets',
    price: 2500
  },
  {
    _id: ObjectId('67fdef18aa4d1653a24aa593'),
    name: 'Gaming Mouse',
    category: 'Tech Gadgets',
    price: 1500
  },
  {
    _id: ObjectId('67fdef18aa4d1653a24aa598'),
    name: 'Tablet',
    category: 'Tech Gadgets',
    price: 20000
  }
]

test> // Products in Tech Gadgets OR price < 10000
... db.products.find({
...   $or: [
...     {category: 'Tech Gadgets'},
...     {price: {$lt: 10000}}
...   ]
... });
[
  {
    _id: ObjectId('67fdeee3aa4d1653a24aa58c'),
    name: 'Laptop',
    category: 'Tech Gadgets',
    price: 70000
  },
  {
    _id: ObjectId('67fdeeedaa4d1653a24aa58d'),
    name: 'Smartphone',
    category: 'Tech Gadgets',

```

```

    price: 10000
  },
  {
    _id: ObjectId('67fdeeedaa4d1653a24aa58e'),
    name: 'Books',
    category: 'Stationary',
    price: 500
  },
  {
    _id: ObjectId('67fdef18aa4d1653a24aa58f'),
    name: 'Headphones',
    category: 'Tech Gadgets',
    price: 2500
  },
  {
    _id: ObjectId('67fdef18aa4d1653a24aa590'),
    name: 'Blender',
    category: 'Home Appliances',
    price: 3000
  },
  {
    _id: ObjectId('67fdef18aa4d1653a24aa591'),
    name: 'Notebook',
    category: 'Stationary',
    price: 50
  },
  {
    _id: ObjectId('67fdef18aa4d1653a24aa592'),
    name: 'T-shirt',
    category: 'Fashion',
    price: 800
  },
  {
    _id: ObjectId('67fdef18aa4d1653a24aa593'),
    name: 'Gaming Mouse',
    category: 'Tech Gadgets',
    price: 1500
  },
  {
    _id: ObjectId('67fdef18aa4d1653a24aa594'),
    name: 'Desk Lamp',
    category: 'Home Decor',
    price: 1200
  },
  {
    _id: ObjectId('67fdef18aa4d1653a24aa595'),
    name: 'Backpack',
    category: 'Accessories',
    price: 1800
  },
  {
    _id: ObjectId('67fdef18aa4d1653a24aa596'),
    name: 'Shoes',
    category: 'Fashion',
    price: 2500
  },
  {
    _id: ObjectId('67fdef18aa4d1653a24aa597'),
    name: 'Water Bottle',
    category: 'Utilities',
    price: 300
  },
  {
    _id: ObjectId('67fdef18aa4d1653a24aa598'),
    name: 'Tablet',
    category: 'Tech Gadgets',
    price: 20000
  }
]

```

```

test> // Products in category "Stationary" AND price < 1000
... db.products.find({

```



```

...   $and: [
...     {category: 'Stationary'},
...     {price: {$lt: 1000}}
...   ]
... });
[
  {
    _id: ObjectId('67fdeeedaa4d1653a24aa58e'),
    name: 'Books',
    category: 'Stationary',
    price: 500
  },
  {
    _id: ObjectId('67fdef18aa4d1653a24aa591'),
    name: 'Notebook',
    category: 'Stationary',
    price: 50
  }
]

test> // Not Stationary
... db.products.find({
...   category: {$not: {$eq: 'Stationary'}}
... });
[
  {
    _id: ObjectId('67fdeee3aa4d1653a24aa58c'),
    name: 'Laptop',
    category: 'Tech Gadgets',
    price: 70000
  },
  {
    _id: ObjectId('67fdeeedaa4d1653a24aa58d'),
    name: 'Smartphone',
    category: 'Tech Gadgets',
    price: 10000
  },
  {
    _id: ObjectId('67fdef18aa4d1653a24aa58f'),
    name: 'Headphones',
    category: 'Tech Gadgets',
    price: 2500
  },
  {
    _id: ObjectId('67fdef18aa4d1653a24aa590'),
    name: 'Blender',
    category: 'Home Appliances',
    price: 3000
  },
  {
    _id: ObjectId('67fdef18aa4d1653a24aa592'),
    name: 'T-shirt',
    category: 'Fashion',
    price: 800
  },
  {
    _id: ObjectId('67fdef18aa4d1653a24aa593'),
    name: 'Gaming Mouse',
    category: 'Tech Gadgets',
    price: 1500
  },
  {
    _id: ObjectId('67fdef18aa4d1653a24aa594'),
    name: 'Desk Lamp',
    category: 'Home Decor',
    price: 1200
  },
  {
    _id: ObjectId('67fdef18aa4d1653a24aa595'),
    name: 'Backpack',
    category: 'Accessories',
    price: 1800
  }
]

```

```

    },
    {
      _id: ObjectId('67fdef18aa4d1653a24aa596'),
      name: 'Shoes',
      category: 'Fashion',
      price: 2500
    },
    {
      _id: ObjectId('67fdef18aa4d1653a24aa597'),
      name: 'Water Bottle',
      category: 'Utilities',
      price: 300
    },
    {
      _id: ObjectId('67fdef18aa4d1653a24aa598'),
      name: 'Tablet',
      category: 'Tech Gadgets',
      price: 20000
    }
  ]
}

```

```

test> // Not Tech Gadgets and price < 50000
... db.products.find({
...   $nor: [
...     {category: 'Tech Gadgets'},
...     {price: {$gte: 50000}}
...   ]
... });
[
  {
    _id: ObjectId('67fdeeedaa4d1653a24aa58e'),
    name: 'Books',
    category: 'Stationary',
    price: 500
  },
  {
    _id: ObjectId('67fdef18aa4d1653a24aa590'),
    name: 'Blender',
    category: 'Home Appliances',
    price: 3000
  },
  {
    _id: ObjectId('67fdef18aa4d1653a24aa591'),
    name: 'Notebook',
    category: 'Stationary',
    price: 50
  },
  {
    _id: ObjectId('67fdef18aa4d1653a24aa592'),
    name: 'T-shirt',
    category: 'Fashion',
    price: 800
  },
  {
    _id: ObjectId('67fdef18aa4d1653a24aa594'),
    name: 'Desk Lamp',
    category: 'Home Decor',
    price: 1200
  },
  {
    _id: ObjectId('67fdef18aa4d1653a24aa595'),
    name: 'Backpack',
    category: 'Accessories',
    price: 1800
  },
  {
    _id: ObjectId('67fdef18aa4d1653a24aa596'),
    name: 'Shoes',
    category: 'Fashion',
    price: 2500
  },
  {

```

```
    _id: ObjectId('67fdef18aa4d1653a24aa597'),  
    name: 'Water Bottle',  
    category: 'Utilities',  
    price: 300  
  }  
]  
  
test> db.products.deleteOne({name: 'Laptop'});  
{ acknowledged: true, deletedCount: 1 }  
  
test> db.products.deleteMany({ category: 'Fashion' });  
{ acknowledged: true, deletedCount: 2 }
```

CONCLUSION:-

In conclusion, this assignment demonstrates the effective use of MongoDB for performing CRUD (Create, Read, Update, Delete) operations on a product database. It included creating collections, inserting documents, retrieving specific records using filters and logical operators, updating fields, and deleting data conditionally. The assignment highlights MongoDB's flexibility in handling document-based data and showcases its efficiency for managing real-world product inventories in a NoSQL environment.